Evaluating Large Language Models for Automated Exploratory Data Analysis and Natural Language-Based Data Querying

Sidharth Malik
Department of Computer Applications
Chandigarh University
Gharuan, Mohali, Punjab, India
26MCD10110@cuchd.in

***Abstract*—Large language models (LLMs) have been recently positioned as an accessibility layer between analysts/writers and structured data, enabling natural language queries to yield charts, summaries, or executable code without requiring hand-written SQL or Python. This paper evaluates and contrasts the state-of-the-art in LLMs-based systems for two related but distinct tasks: automated exploratory data analysis, and natural language-based querying (typically Text-to-SQL). Surveying the literature, including the benchmarks (Spider, Spider 2.0, BIRD) and systems (LIDA, PandaSAI, DAIL-SQL) under consideration, the paper aims to contribute a discussion of what gets measured, what is not measured, and how the precision of these LLM-based systems collapses when transitioning from clean academic schemas to messy enterprise databases. What gets evaluated and how is discussed using a 5-dimensional comparison framework, while the review highlights a number of research challenges, ranging from the "performance cliff" of LLMs on more realistic data, to the limitations of existing benchmarks and evaluation practices. The paper ultimately argues that, at least in the near term, LLMs should be viewed as copilots for human or symbolic analysis rather than standalone exploratory data analysis systems, and discusses directions for benchmarking, hybrid symbolic/LLM approaches, and human-in-the-loop evaluation.**

I. Introduction

The amount of data being collected by organizations greatly exceeds the amount being analyzed by trained analysts. Business intelligence dashboards only answer the questions their designers anticipate, and writing ad-hoc SQL or Python queries continues to require technical proficiency. The advent of large language models with strong code-generation capabilities has renewed interest in making a system where a user could ask a question in natural language and receive an answer in the form of a SQL query, Python snippet, or chart plot over their private data.

This paper considers two tasks, which are related but differ significantly in their technical details. The first, natural language-based querying, is a translation task: given a question and a database schema description, generate an executable query (either SQL for relational databases or pandas code for CSV files) that would return a result. The second task, automated exploratory data analysis, is an open-ended generation task: given a database, decide what to visualize/examine to better understand its properties, and generate either the corresponding analysis code or visualizations directly.

A system can have strengths in one task and weaknesses in the other: an LLM might be able to translate natural language questions to SQL queries with high accuracy, but it might not be able to determine what to visualize for an unfamiliar database. This paper therefore discusses the following research questions:

RQ1: How accurate is the translation of user questions to executable queries as database size and complexity increases? (Section VI)

RQ2: How capable are systems at performing open-ended, automated exploratory data analysis? (Section VI)

RQ3: What are the major benchmarks and metrics for these tasks, and what are their limitations? (Section IV)

RQ4: What are the major practical limitations and risks of such systems? (Section VII)

Section II provides the background and related work; it discusses the history and state-of-the-art for both exploratory data analysis and natural language interfaces to databases. Section III reviews the major LLM-based systems for automated Querying (Text-to-SQL) and for automated Exploratory Data Analysis (EDA)/report writing. Section IV discusses the major benchmarks and metrics for the tasks reviewed in Section III. Section V provides a framework for comparative discussion, and Sections VI and VII discuss the findings and the limitations, respectively. Finally, Section VIII highlights directions for future work and Section IX concludes the paper.

II. Background and Related Work

To provide the context for this survey, this section briefly reviews the literature on automated exploratory data analysis and natural language interfaces to databases.

A. Exploratory Data Analysis

Exploratory data analysis (EDA), in its canonical form, is an examination of a given dataset to discover its properties and potential insights. Today, EDA is typically performed manually using visualization and statistical tools, such as pandas, Tableau, or Power BI. A major time expenditure in the process comes from the analyst having to select what to visualize: for a given set of hundreds of columns, it is necessary to decide what to

plot and in what order to visualize it. Automatically determining the most interesting visualization to make given a dataset is the defining characteristic of the task of automated EDA.

B. Natural Language Interfaces to Databases and Text-to-SQL

Natural language interfaces to databases (NLIDBs) have a long history, dating back to early rule-based systems and progressing to modern encoder-decoder sequence-to-sequence models, which are typically trained on question-query paired data. With the advent of large-scale cross-domain benchmarks such as Spider, the focus shifted towards systems' ability to generalize: a Text-to-SQL system should be able to handle questions about new databases that it has not seen during training. The rise of large language models capable of few-shot and zero-shot code generation changed the paradigm again: today, the dominant approach is to fine-tune a language model or to prompt a strong foundation model to generate the SQL query directly, potentially with some additional schema instructions or by employing decomposition and self-correction steps.

C. LLMs as a New Paradigm for Data Analysis

What makes foundation models and their ability to generate code different from previous approaches is that their capabilities can be leveraged not only for query translation but also for writing analysis scripts. An automated EDA system typically follows an iterative approach: it generates candidate code, executes it in an execution environment to obtain results, and updates the code based on the results until obtaining satisfactory results. Such an approach underlies most approaches to automated EDA (reviewed in Section III).

III. Survey of LLM-Based Systems

There are three broad approaches for building systems capable of performing automated data analysis: (1) prompting-based systems for translating natural language questions to executable queries or scripts, (2) automated EDA systems for generating suggestions for what to visualize given a database, and (3) end-to-end systems which attempt to perform various stages of data science pipeline using a combination of language models and execution environments.

A. Text-to-SQL Systems

Prompting-based systems for Text-to-SQL typically attempt to find the best representation for the schema and the best way to represent the examples for the model to learn from. For example, DAIL-SQL evaluates various prompt and few-shot example selection approaches for closed-vocabulary models and achieves an execution accuracy of over 86% on the Spider leaderboard using GPT-4, with further improvements possible via self-consistency voting [8]. Another approach is to decompose the question into schema linking and subquestion generation components, with methods such as Decomposed-SQL achieving an execution accuracy of 72.4% on the Spider benchmark [10]. Self-consistency and multi-agent approaches also improve accuracy: multiple models can be used to generate answers which are then scored against a reference or execution result and the best response is selected [9]; similarly, a dedicated knowledge-sources model can be fine-tuned to provide external knowledge when the main model is insufficient, as in Knowledge-to-SQL [11]. Search-based approaches attempt to find a query tree using heuristic and self-rewarding search, such as in SQL-o1 [7]. Finally, distillation approaches such as SENSE aim to train smaller open-weight models using a mixture of strong teacher models, achieving similar performance to prompted closed-source models on Spider at a significantly lower computational cost [6].

B. Automated EDA and Visualization Systems

LIDA is an automated visualization system developed by Microsoft Research, which employs a modular approach to visualization generation: the Summarizer analyzes the raw database and outputs a natural language description, the Goal Explorer generates potential question-visualization rationale triples, the VizGenerator creates the plotting code and executes it, and the Infographics module applies transformations to the obtained plots [5], [15]. This approach is designed to allow "EDA for free" – the user does not have to ask a particular question; rather, the system suggests what can be interesting to visualize given a database. Another approach is to wrap an existing Python library, such as pandas, into a large language model agent – this is the approach of PandasAI, which allows the user to input natural language queries, which are then converted to pandas code by an LLM [19]. More recent approaches use multiple agents to achieve end-to-end "data storytelling" – a question-answering agent is combined with a chart generation agent and a chart explanation agent to produce a full analysis report consisting of charts and natural language paragraphs [12], [14].

C. End-to-End and Agentic Data Science Systems

A number of recent works have aimed to build an end-to-end system capable of performing multiple stages of the data science pipeline. A survey of such systems identifies three major layers: foundational capabilities (automated data understanding, simple EDA), advanced capabilities (complex visualization, reasoning), and infrastructure layer (multimodal synthesis, safety) [10], [13].

IV. Benchmarks and Evaluation Methodologies

A. Spider and Spider 2.0

Spider is a popular cross-domain Text-to-SQL benchmark, which contains around 200 databases and 138 domains, with approximately 7000 training and 1034 development question-query pairs, evaluated using execution accuracy [1], [8]. Spider 2.0 is a more recent benchmark, aimed at reducing the gap between research and industry use cases. Its databases are larger and more complex than those in the original Spider: they contain over 3000 columns and use a mix of SQL dialects, including BigQuery and Snowflake, and queries often require multiple steps, including analytical functions and transformations [3].

B. BIRD

BIRD is a benchmark designed to evaluate an LLM's ability to answer questions about real-world databases. It contains 95 databases (totaling around 33.4 GB of data) from 37 professional domains and has 12751 text-SQL pairs, with an emphasis on natural language questions about messy and dirty data [2], [9]. BIRD also introduces the Valid Efficiency Score, which measures the efficiency of a query (in terms of execution time).

C. Benchmark Reliability Concerns

A 2026 re-annotation study serves as an important cautionary note regarding the use of benchmarks. By reviewing a subset of BIRD and Spider 2.0-Snow annotations, the authors identified roughly 52.8% and 66.1% of annotations, respectively, which contained errors. When leading Text-to-SQL models were re-evaluated on the corrected benchmark, their relative performance changed by as much as 31%, with individual models' ranks changing by up to three positions [4]. Any evaluation based on a published leaderboard therefore must be considered as a ranking relative to other approaches rather than an objective assessment of practical accuracy.

D. Visualization and EDA-Specific Evaluation

Unlike query answering, where a correct SQL query has an unambiguous result, evaluation of visualization requires different approaches. LIDA's evaluation on the Vega repository consists of 57 datasets, and uses both a visualization error rate (percentage of charts that failed to execute) and a visualization quality score, predicted by GPT-4 on a 6-axis scale: code accuracy, data transformations, goal compliance, visualization type, data encoding, and aesthetics [5]. The Drawing Pandas benchmark focuses on the narrower task of measuring whether an LLM can generate correct plotting code from a natural language description [14]. Similarly to automated EDA systems, it evaluates the quality of charts by using an LLM as the judge, which presents potential pitfalls discussed in Section VII.

E. Summary of Metrics

- Execution Accuracy (EX): whether the result of the executed query matches the gold result.
- Test-Suite Accuracy (TS): the percentage of queries for which execution accuracy is correct across different databases (as opposed to a single one).
- Valid Efficiency Score (VES): the efficiency score for a query, balancing its correctness and execution time (relevant for BIRD).
- Visualization/code error rate: the percentage of visualizations or analysis code that failed to execute.
- LLM-judged quality score: an aggregate score predicted by an LLM for the quality of generated charts or text.
- Latency and token cost: the number of tokens processed and the execution time, especially for multi-agent and self-correcting approaches.

V. A Proposed Comparative Evaluation Framework

Given the different strengths and weaknesses of the existing systems, this paper uses a 5-dimensional comparative evaluation approach. It can be applied to a student project comparing two or three tools on a single database.

A. Correctness

Is the result the same as what a human analyst would return? More formally, does the execution of the returned query or analysis code produce the same result as the gold standard? String comparison of the generated query to the gold one is known to have issues in cases when equivalent but differently formulated queries exist, hence execution accuracy, when available, is the preferred metric.

B. Robustness

Is the accuracy stable under various modifications of the input? Does the system fail catastrophically when the question is rephrased or when column names in the database do not match the question text? Spider-Syn, Spider-DK, Spider-Realistic and other variants of the benchmark aim to evaluate the system's robustness to realistic questions [9].

C. Efficiency

How many tokens does a single query or analysis take, and what is the overall execution time? For multi-agent systems, this is relevant to each individual agent executed as part of the analysis. Lower token count and faster execution time are better, although it has to be balanced against accuracy.

D. Interpretability

Is it possible for a layperson to review the system's execution and verify that it did not do anything unreasonable? Can they look at the generated SQL query or Python code and see that it follows logical steps, or is the system a "black box" which simply provides an answer? Approaches which involve the user in the loop or which show the executed code, such as the automated EDA systems reviewed in Section III, have an advantage in this regard.

E. Trustworthiness and Safety

Does the system make up columns or databases which do not exist? Does it execute the generated SQL code in an isolated environment where it cannot modify production data? Finally, does the system protect itself and the user from prompt injection attacks when the database contains user input which can be manipulated? This dimension also considers the safety of the data: does the system require the user to provide schema and example data to a third-party API?

An example application of the framework is a student project, where one would take an example database and run the same set of questions through two or three systems (for example, a direct prompt to a general-purpose LLM, a specialized tool such as PandasAI, and a fine-tuned smaller LLM) and score each according to the five dimensions, highlighting differences

between the systems.

VI. Comparative Discussion of Findings

Several trends can be observed when reviewing the surveyed works. First, prompting-based closed-source models continue to dominate in terms of accuracy on the standard benchmarks, but fine-tuned smaller open models can perform nearly as well on the same benchmarks while being significantly more cost-effective to deploy: for example, SENSE-7B and SENSE-13B achieve roughly 83-84% execution accuracy on the development set of Spider, compared to around 86% from prompted GPT-4, and are competitive with other prompted alternatives, albeit with somewhat lower accuracy on the BIRD benchmark (roughly 52-56%) compared to Spider (roughly 83%) [6], [8].

Second, and more importantly, the accuracy on the enterprise-focused Spider 2.0 is significantly lower than on the original Spider 1.0: the same GPT-4o which scores 86.6% on Spider 1.0 only achieves around 10.1% accuracy on the enterprise-scale multi-dialect version of Spider 2.0 [3]. This is the most concrete illustration of a "performance cliff" between research and production mentioned in one of the recent industry analyses [5-alt]. Any practical deployment of a Text-to-SQL system should therefore be evaluated on a realistically complex database, not on a smaller academic benchmark. Similar findings are expected to apply to EDA systems, although such systems have not been tested on analogous realistic benchmarks yet.

Other findings relate to the benchmark reliability and evaluation practices. There is growing concern that benchmarks, particularly Spider and BIRD which have been in use for several years, may have significant contamination with training data, and recent re-annotation efforts have revealed that roughly every second annotation in Spider 2.0-Snow and BIRD contained errors [4]. While noisy labels are known to affect the accuracy of large language models disproportionately, the effect on the relative rank of different approaches is also significant: when the authors of the re-annotation study re-scored the Spider 2.0-Snow benchmark, the relative accuracy of different models changed by up to 31% on average and individual models' ranks changed by as much as three positions [4]. Thus, any ranking based on a published leaderboard should be treated with caution. Finally, automated EDA systems typically evaluate chart generation using an LLM as a judge, which is susceptible to the same issues of inaccurate scoring mentioned above [5], [14].

VII. Challenges and Limitations

- Hallucinated schema: references to non-existent columns or tables, particularly in larger databases.
- Execution safety: avoiding injection attacks or unintended database modification if the generated SQL code is executed directly.
- Benchmark contamination: the presence of training data in the benchmark, which can artificially inflate model performance.
- LLM-as-judge circularity: using another LLM to score the quality of the results can lead to inaccurate scoring if the two models share similar biases.
- Cost of self correction: pipelines that use multiple LLMs to achieve higher accuracy, such as self-consistency voting, can be prohibitively expensive for practical use cases.
- Data privacy: in some cases, a user may not be able or willing to provide the schema and example data to a third-party API.

VIII. Future Directions

- Hybrid approaches where an LLM answers questions but its results are validated by a symbolic database query planner or other trusted execution environment.
- Benchmarks with more reliable annotations and designed to reflect practical use cases rather than clean academic data.
- Human-in-the-loop evaluation methodologies, including standardized tests for real-world ambiguous questions, where a human analyst has to determine the most reasonable chart or question to ask.
- Smaller, fine-tuned models for specific domains or tasks, following the distillation approach pioneered by the SENSE project.
- Unifying systems where an agent performs both open-ended automated EDA (asking what to visualize) and responding to specific natural language questions.
- Agentic systems which utilize multiple models to plan and execute queries, potentially using reasoning similar to SQL interpreters but implemented in code.

IX. Conclusion

Large language models have enabled significant progress in automated data analysis, with rapid improvements in performance on standard benchmarks such as Spider and BIRD. At the same time, the limitations of such systems, reflected in their inability to handle realistic use cases and the difficulties of evaluating their performance accurately, argue for their restricted deployment as standalone analysts. The most reasonable short-term approach for adopting such systems is therefore to use them as assistants to human or symbolic analysis rather than as standalone tools. The paper highlights future research directions, including hybrid neuro-symbolic approaches, more realistic benchmarks, and human-in-the-loop evaluation of automated EDA systems. For current deployment is as an analyst's copilot: a system which drafts queries, suggests candidate visualizations, and proposes exploration goals, and which is checked by a human, or a symbolic verification layer before it is trusted. Future improvements in benchmark integrity, hybrid verification, and human-in-the-loop evaluation will determine how far that trust can be extended.

References

[1] Yu, T., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. https://yale-lily.github.io/spider

[2] Li, J., et al. (2023). BIRD: A Big Bench for Large-Scale Database Grounded Text-to-SQL Evaluation. https://bird-bench.github.io

[3] Lei, F., et al. (2024). Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. arXiv:2411.07763. https://spider2-sql.github.io/

[4] Jin, R., et al. (2026). Text-to-SQL Benchmarks are Broken: An In-Depth Analysis of Annotation Errors. CIDR'26. https://www.vldb.org/cidrdb/papers/2026/p5-jin.pdf

[5] Dibia, V. (2023). LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics Using Large Language Models. Proceedings of the 61st Annual Meeting of the ACL (System Demonstrations). https://doi.org/10.18653/v1/2023.acl-demo.11; https://microsoft.github.io/lida/

[5-alt] Mysore, V. (2026). The Text-to-SQL Performance Cliff (2026): Why "Natural Language to SQL" Breaks. Medium. https://medium.com/@visrow/the-text-to-sql-performance-cliff-2026-why-natural-language-to-sql-breaks-a7281a23dbea

[6] Yang, J., et al. (2024). Synthesizing Text-to-SQL Data from Weak and Strong LLMs (SENSE). arXiv:2408.03256

[7] SQL-o1: A Self-Reward Heuristic Dynamic Search Method for Text-to-SQL. (2025). arXiv:2502.11741

[8] Gao, D., et al. (2023). Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation (DAIL-SQL). arXiv:2308.15363

[9] SIMBA UQ: Similarity-Based Aggregation for Uncertainty Quantification in Large Language Models. (2025). arXiv:2510.13836

[10] SQLFixAgent: Towards Semantic-Accurate Text-to-SQL Parsing via Consistency-Enhanced Multi-Agent Collaboration. (2024). arXiv:2406.13408

[11] Knowledge-to-SQL: Enhancing SQL Generation with Data Expert LLM. (2024). arXiv:2402.11517

[12] LLM-Powered Visualizations and Narratives from Natural Language Queries. (2025). CEUR-WS Vol-4099. https://ceur-ws.org/Vol-4099/ER25_PAD_Nascimento.pdf

[13] Measuring Data Science Automation: A Survey of Evaluation Tools for AI Assistants and Agents. (2025). arXiv:2506.08800

[14] Drawing Pandas: A Benchmark for LLMs in Generating Plotting Code. (2024). arXiv:2412.02764

[15] A Review on Large Language Models for Visual Analytics. (2025). arXiv:2503.15176

[16] LLM-Based Data Science Agents: A Survey of Capabilities, Challenges, and Future Directions. (2025). arXiv:2510.04023